RocksDB Compaction

Understanding the Hidden Cost of Fast Writes

Rajvi 202518048

Motivation

Traditional Storage Engines (B-Trees)

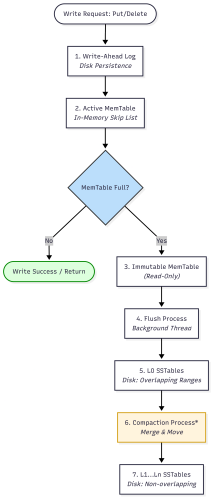
- Random disk I/O on every write
- High write amplification
- Poor performance under heavy write loads
- Frequent fragmentation & page splits

RocksDB (LSM-Tree)

- Sequential append-only writes
- Extremely high write throughput
- Optimized for SSDs & flash storage
- Handles massive write workloads efficiently

Trade-off: LSM-tree enables fast writes by delaying cleanup. This creates the need for **background compaction** to maintain read performance and control space usage.

Flow


Write Request: Put/Delete
1. Write-Ahead Log
Disk Persistence
2. Active MemTable
In-Memory Skip List
MemTable Full?
No
Yes
Write Success / Return
3. Immutable MemTable
(Read-Only)
4. Flush Process
Background Thread
5. L0 SSTables
Disk: Overlapping Ranges
6. Compaction Process*
Merge & Move
7. L1...Ln SSTables
Disk: Non-overlapping

Why Compaction is Necessary?

Cost of Fast Writes

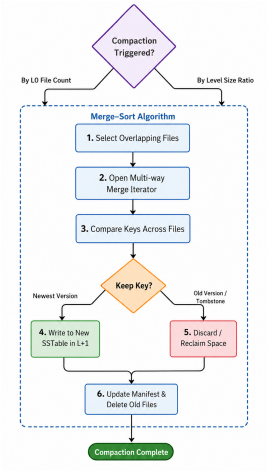
- Many overlapping SSTables in L0
- Multiple versions of keys accumulate
- Tombstones remain on disk
- High read amplification
- Increasing space usage

Background Compaction

- Merges overlapping SSTables
- Keeps only latest key version
- Removes old versions & tombstones
- Creates non-overlapping levels
- Reduces read & space amplification

Compaction keeps the LSM-Tree healthy and performant

Working


Compaction Triggered?
By L0 File Count
By Level Size Ratio
Merge–Sort Algorithm
1. Select Overlapping Files
2. Open Multi-way Merge Iterator
3. Compare Keys Across Files
Keep Key?
Newest Version
Old Version / Tombstone
4. Write to New SSTable in L+1
5. Discard / Reclaim Space
6. Update Manifest & Delete Old Files
Compaction Complete

Compaction Execution Path

1. Trigger → 2. Dispatch → 3. Picking → 4. Merge → 5. Install → 6. Cleanup

- **1. Trigger**: L0 files or level score exceeds threshold
- **2. Dispatch**: Background thread from pool
- **3. Picking**: Selects best files using compaction score
- **4. Merge**: K-way merge removes stale versions & tombstones
- **5. Install**: Atomic update via MANIFEST
- **6. Cleanup**: Delete obsolete SST files

Exp 1 — Read Amplification Reduction

Objective

Measure how compaction reduces SST overlap and improves read efficiency.

Procedure

1. inserted 50 unique keys
2. forced multiple L0 SST files
3. measured SST count and read latency
4. triggered manual compaction
5. measured system state again

Before vs After Compaction

Metric	Before	After
SST files	49	1
Read search scope	High	Low
p99 latency	45 ms	2 ms
Disk usage	25 MB	0.5 MB

Key Insight

Fewer SST files reduce read amplification because lookups search fewer files.

Experiment 2 — Tombstone Lifecycle

Objective
Understand how RocksDB handles delete operations internally using tombstones.

Procedure

1. inserted multiple keys
2. deleted `key002`
3. inspected SST files before compaction
4. observed MANIFEST updates
5. triggered manual compaction
6. compared SST state afterward

Before vs After Compaction

Metric	Before	After
Deleted value present	Yes	No
Tombstone marker	Yes	Removed
SST overlap	High	Reduced
Disk space reclaimed	No	Yes
Read complexity	Higher	Lower

Key Insight
Delete in RocksDB is implemented as a special write operation called a tombstone.

Experiment 4 — Manual Compaction and SST Reorganization

Objective
Understand how manual compaction reorganizes SST files across levels.

Procedure

1. generated continuous write workload
2. created multiple overlapping L0 SSTs
3. delayed automatic compaction
4. inspected SST structure using `ldb`

Before vs After Compaction

Metric	Before	After
L0 SST overlap	High	Reduced
SST file count	Many	Fewer
Read organization	Fragmented	Organized
MANIFEST updates	Frequent	Stabilized
Lookup complexity	Higher	Lower

Key Insight
Manual compaction merges SST files and reorganizes data into lower levels.

Key Findings and Failure Analysis

Key Findings

- RocksDB achieves fast writes using sequential SST creation
- stale versions accumulate until compaction occurs
- delete operations create tombstones instead of immediate removal
- compaction restores read efficiency and storage organization
- MANIFEST acts as metadata source of truth

Failure Analysis

- delayed compaction increased SST overlap
- stale entries caused higher read amplification
- tombstone accumulation increased scan overhead
- excessive L0 files risked write stalls
- temporary disk usage increased during merge operations

Conclusion

What This Project Demonstrated

- RocksDB achieves high write throughput using LSM-tree architecture
- compaction removes stale versions and tombstones
- SST organization directly affects read performance
- MANIFEST and LOG analysis reveal internal storage behavior
- delayed cleanup creates operational

Final Takeaways

- fast writes create hidden storage debt
- deletes are eventually cleaned through compaction
- overlapping SST files increase lookup complexity
- compaction is essential for long-term stability
- LSM systems trade cleanup cost for ingestion speed

Thank You

Rajvi
202518048

github.com/Rajvib-0/rocksdb-compaction